

Repaso rápido

Variables, condicionales y bucles

1.1 Variables, tipos y arrays

Java es fuertemente tipado: cada variable declara su tipo, y ese tipo no cambia durante la ejecución.

Categoría	Ejemplos	Guarda
Primitivo	int, double, boolean, char	el valor directamente
Referencia	String, Persona, ArrayList<T>	una dirección al objeto

```
int[] numeros = {2, 4, 6, 8}; // tamaño fijo: 4 huecos, ni uno más
System.out.println(numeros[0]); // 2, el primer elemento
```

Esa limitación de tamaño fijo es justo la que resuelven las colecciones, en el siguiente apartado.

1.1 Tipos numéricos

No todos los números son iguales: cada tipo numérico ocupa un tamaño distinto y eso limita su rango y si admite decimales.

Tipo	Tamaño	¿Decimales?	Cuándo usarlo
int	32 bits	No	el tipo por defecto para enteros
long	64 bits	No	cuando un int se queda corto
double	64 bits	Sí	el tipo por defecto para decimales
float	32 bits	Sí	decimales, ahorrando memoria

```
long poblacionMundial = 8_100_000_000L; // la "L" indica long
float precioAproximado = 19.99f;         // la "f" indica float
```

1.1 Cadenas de texto (String)

Un String es inmutable: ningún método lo modifica, todos devuelven un String nuevo.

Método	Qué hace	Ejemplo
length()	número de caracteres	"Hola".length() → 4
charAt(i)	carácter en la posición i	"Hola".charAt(0) → 'H'
substring(i,j)	trozo de texto entre i y j	"Hola".substring(1,3) → "ol"
toUpperCase()/toLowerCase()	mayúsculas / minúsculas	"Hola".toUpperCase() → "HOLA"
trim()	quita espacios de los extremos	" Hola ".trim() → "Hola"
equals(otro)	compara el contenido	"Hola".equals("hola") → false

1.2 Condicionales

if/else y switch deciden qué bloque de código se ejecuta según una condición booleana.

```
int edad = 17;

if (edad >= 18) {
    System.out.println("Es mayor de edad");
} else {
    System.out.println("Es menor de edad");
}
```

Para comparar dos String nunca uses ==: usa siempre .equals(). Con primitivos (int, double, boolean) sí puedes usar ==.

1.2 If anidado y cadenas else if

Cuando hay más de dos posibilidades, los if se anidan: una cadena de else if evita acumular llaves de más.

```
int nota = 6;

if (nota >= 9) {
    System.out.println("Sobresaliente");
} else if (nota >= 7) {
    System.out.println("Notable");
} else if (nota >= 5) {
    System.out.println("Suficiente");
} else {
    System.out.println("Insuficiente");
}
```

Cada else if solo se comprueba si los anteriores han sido falsos, y en cuanto uno se cumple, el resto ya no se evalúa.

1.3 Bucles: for, while y do-while

```
for (int i = 0; i < 4; i++) {  
    System.out.println("Vuelta número " + i);  
}  
  
int intentos = 0;  
while (intentos < 3) { intentos++; }  
  
int numero;  
do {  
    numero = leerNumero();  
} while (numero < 0);
```

La diferencia: un while puede no ejecutarse nunca; un do-while se ejecuta siempre al menos una vez.

1.3 For-each

El for-each (for (Tipo elemento : coleccion)) es el que más vas a usar para recorrer arrays, listas y colecciones sin manejar índices a mano.

```
int[] numeros = {2, 4, 6, 8};  
  
for (int n : numeros) {  
    System.out.println(n);  
}
```

A partir de aquí preferimos for-each y, más adelante, streams frente al for clásico con índice.